见 途 Jiantu

面向视障者与陪同者的 AI 出行准备与环境观察助手

技 术 文 档

参赛赛项：全国高校 AI Web 开发挑战赛（省赛 • 作品题）

参赛组别：B 组

团队名称：菁核

所在院校：三江学院

作品形态：PC 端 Web 应用（React + TypeScript + 本地 AI 服务）

2026 年 11 月

目 录

第一章 项目概述.....3
1.1 项目背景.....3
1.2 选题定位：从“行中导航”到“行前准备”.....3
1.3 开发目的与项目意义.....4
1.4 本文结构.....4
第二章 需求分析.....5
2.1 目标用户与使用场景.....5
2.2 功能需求分析.....5
2.3 非功能需求分析.....6
第三章 系统架构设计.....7
3.1 总体架构.....7
3.2 前端架构与页面组织.....8
3.3 服务层与接口设计.....8
3.4 数据与配置管理.....9
第四章 AI 技术方案.....10
4.1 技术选型与选型理由.....10
4.2 Agent：受限工具循环.....10
4.3 RAG：先检索，后生成，宁缺毋滥.....13
4.4 多模态：视觉分级与语音策略.....15
4.5 流式交互：让等待变成过程.....17
4.6 AI 安全与可控性.....19
第五章 核心功能实现.....20
5.1 出行助手.....20
5.2 室内寻物.....22
5.3 室外感知.....23
5.4 无障碍知识问答.....24
5.5 无障碍设计与优雅降级.....25
第六章 测试与部署.....26
6.1 测试策略.....26
6.2 室内模型的评测口径.....26
6.3 浏览器端实测.....27
6.4 部署、运行与性能.....27
第七章 创新点说明.....29
7.1 技术创新点.....29
7.2 应用创新点.....29
第八章 总结与展望.....30
8.1 项目成果.....30
8.2 局限与如实声明.....30
8.3 未来规划.....30

第一章 项目概述

1.1 项目背景

出门这件事，对多数人只需要一个决定：打开地图，看一眼路线，抬脚就走。但对视障者与他们的家人，一次出行往往意味着一整套准备：这条路上有没有台阶，换乘站台的电梯今天是否运行，目的地的入口开在哪一侧，附近有没有可以歇脚的地方。这些问题不解决，路线再精确也无法变成安心的出发。

我们在与视障同学和志愿者的交流中发现，现有出行产品几乎都把力气花在“行走的过程中”：实时定位、语音导航、转向提醒。而真正消耗心力的环节，恰恰发生在出发之前。一个人要在没有视觉信息的情况下，把一条两公里外的路线在脑中预演一遍，靠的不是导航播报，而是一次次询问与试错。

与此同时，大模型的能力已经足以承担“把复杂信息整理成人能用的判断”这件事。但把它直接塞进无障碍场景是有风险的：模型会编造站名，会把“约两公里”说成“很近”，会在没有依据时给出肯定结论。对一个看不清路的人来说，一句编造的“前方无障碍”比沉默危险得多。

这正是见途的起点。我们想让 AI 承担整理与解释的工作，同时用工程手段约束它不许越过事实的边界：查得到才说，查不到就说查不到；能比较就摆出依据，不能比较就说明为什么。

1.2 选题定位：从“行中导航”到“行前准备”

见途没有做一款导航软件。我们把产品定位在出行链条的前一段：帮用户把一次出行“想清楚”，而不是在行进中替用户“带路”。定位一旦定下来，其余做法就没得挑了。

最直接的变化是输出的东西不一样。用户确认起点与终点后，见途并行查询步行与公交路径，把结果整理成可比较的方案，按勾选的偏好排序，再附一份行前核实清单。清单里每一项都标着“需自行向场所核实”，系统从不声称已经替用户验证过台阶或电梯。

这一点看着琐碎，其实是产品性质的差别：导航输出指令，见途输出待办事项。指令错了会让人走错路，待办漏了只少问一句。

解释也必须挂在数据上。路线耗时、换乘次数、步行距离全部来自高德返回的结构化结果，页面保留原始卡片；AI 只解释这些数据，任何路线结论都要引用本轮工具返回的方案编号。知识问答同理，回答要落在检索到的条目上，并展示发布机构、资料日期与核验状态。

还有一条：能力边界写在产品里，不写在免责声明里。室外感知页把“分级来自二维框的位置与高度，不是测距”直接印在界面上，语音提示只在连续两帧确认后才播报。未检出的画面会明确显示“不代表道路安全”。

1.3 开发目的与项目意义

开发目的可以拆成三层，越往下越难做。最直接的一层是把不确定性前置：让用户在出门前就知道哪些事需要打电话确认，哪些路段要请人陪同，把“走到一半才发现”的概率降下来。

再往下是让 AI 的判断过程可见。工具调用的每一步、每一条来源、每一次拒答都摊在界面上，用户有权知道结论是怎么来的。最难的一层落在知识问答上：面向家属与志愿者，不提供“经验之谈”，只给可追溯的公开资料，并标注哪些尚未核验。

从社会价值看，见途想回答一个具体问题：当 AI 进入无障碍领域，它该以什么姿态出现。我们的答案是“不做替代者，做准备工作的一部分”。它不替用户判断能不能走，只把“需要判断什么”整理清楚。这种克制也决定了后面的技术选择：凡是会削弱可核实性的能力，宁可不要。

1.4 本文结构

本文按定位、需求、设计、实现、验证的顺序展开，共八章。第四、五章是技术重点，给出判定公式、流程图与时序图；第六、七章提供可复核的测试数据与评测口径。全文图表按章编号，图注置图下，表注置表下。文中数值取自仓库内的代码、测试输出与运行记录，尚未完成的验证直接标注为待办。

第二章 需求分析

2.1 目标用户与使用场景

见途服务三类人，需求各不相同。视障者本人关心“我能不能独立完成这次出行”，陪同者与家属关心“我需要提前准备什么”，志愿者与普通公众关心“遇到需要帮助的人，我该怎么做”。问法不一样，怕的却是同一件事：被含糊地安慰。他们要知道的不是“你很安全”，而是“还有哪些没确认”。

据此我们梳理出四类核心场景：出发前的路线比较与准备、行进中的街面障碍感知、进入陌生室内空间后的物品定位、以及围绕无障碍常识的资料查询。前三类面向具体行动，第四类面向理解与协作。四类场景在界面上对应四个独立页面，彼此互不打断：查询路线时不会被识别画面打断，聊天时也不会清空已有路线结果。

2.2 功能需求分析

系统的功能边界由七个用例界定，参与者为视障用户及其陪同者。用例之间存在包含关系：比较出行方案包含地点候选检索与方案排序，街面障碍感知包含分级判定与语音提示，知识问答包含来源展示与核验状态标注。

需要说明的是，系统刻意不提供两类功能：一是不提供过马路与实时安全通行指令，二是不提供基于实时定位的自动导航。前者涉及无法由二维图像保证判断的安全责任，后者会把“行前准备”错误地延伸为“行中导航”，与产品定位冲突。这两条限制在后端提示词中被写成硬约束，模型无法通过对话绕过。

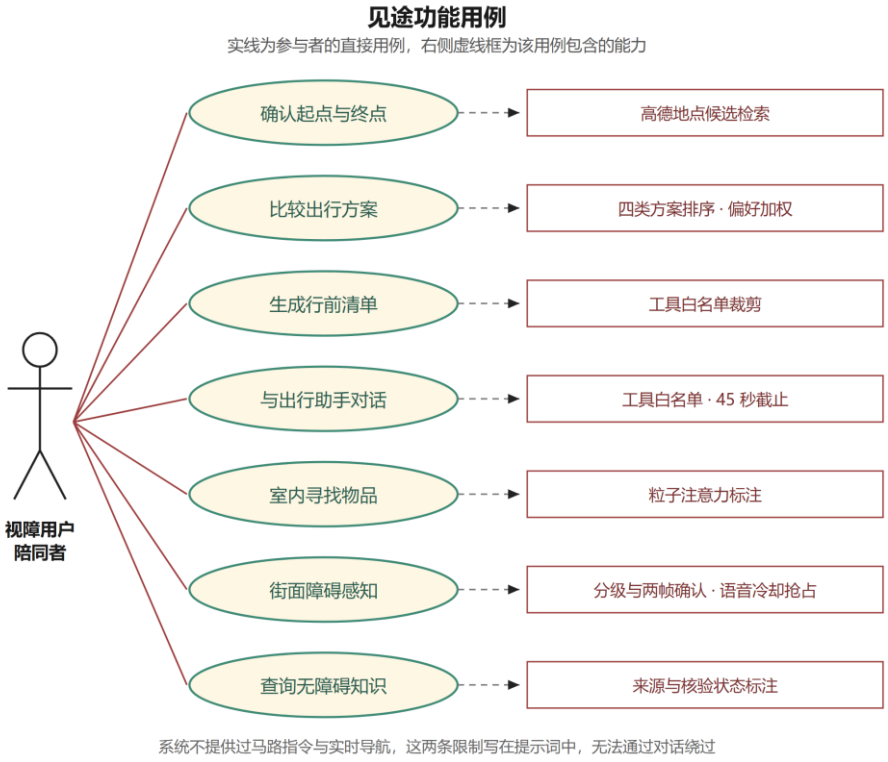


图 2-1 见途功能用例图

2.3 非功能需求分析

无障碍产品的非功能需求，重要性不低于功能本身。五类约束在架构上逐条落实，彼此之间还要让步：可访问性要慢，响应与降级要快，最终以可关闭动效与分级超时调和。

类别	需求描述	落实方式
可访问性	键盘可完整操作，屏幕阅读器可读，支持大字号、高对比、减少动效	原生语义标签与 ARIA 属性；组合框键盘导航；动效统一由 reduced 开关接管
可核实性	任何结论都能追溯到数据来源，无法追溯时明确拒答	工具结果回填原文；来源卡片展示发布机构与日期；检索为空直接拒答
隐私与密钥	密钥不入前端，对话不落盘，图像不上云	密钥仅存于服务端 .env；检测在本机推理；请求体不写日志
响应与降级	单次查询有明确时限，服务异常不导致白屏	前端 30～70 秒分级超时；后端 45 秒硬截止；错误边界与降级文案
成本可控	大模型调用有每日额度上限，超额不影响地图功能	服务端令牌预算守卫；零令牌规则路由分流常见问题

表 2-1 非功能需求与落实方式

第三章 系统架构设计

3.1 总体架构

见途采用“前端渲染、本地服务、端侧推理”的三段式结构，整体划分为四个层次。这样切分不是为了追求层次感，而是由三条硬约束决定的：密钥不能进浏览器，图像不能出本机，模型调用必须可审计。

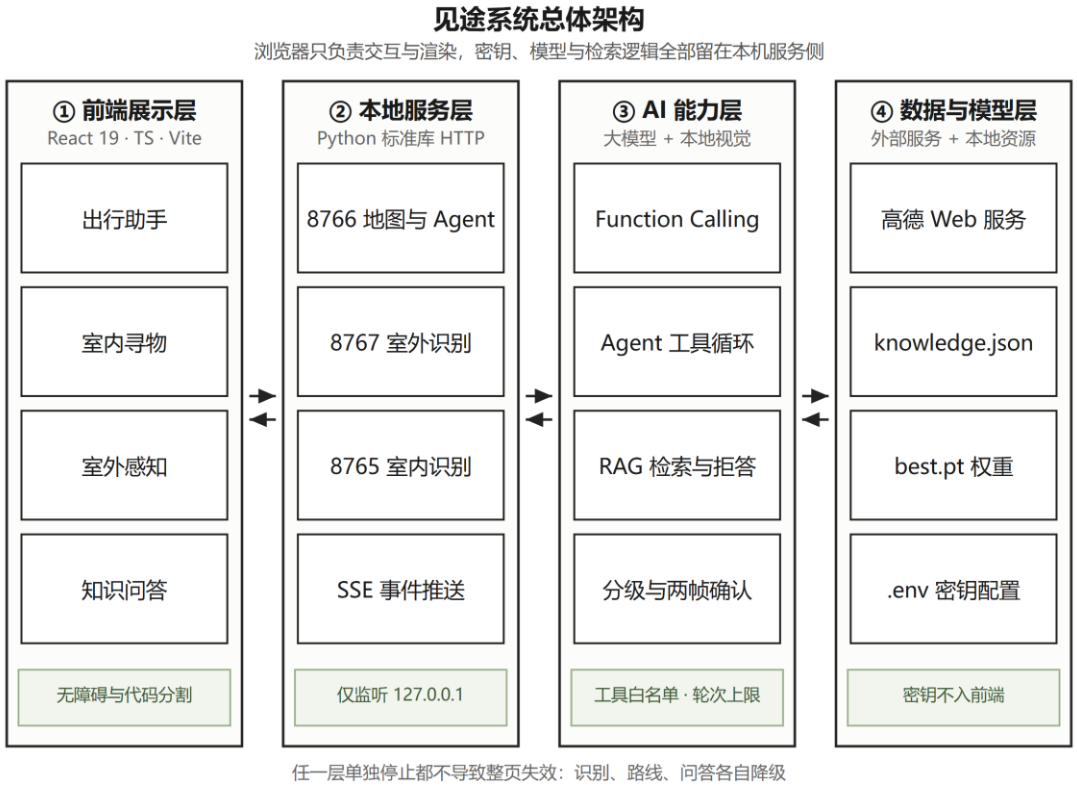


图 3-1 见途系统总体架构

前端展示层只负责交互与渲染，不持有任何密钥。所有需要密钥的请求都经本地服务转发，浏览器侧只与本机 127.0.0.1 通信。

本地服务层由三个独立的 Python 进程构成，全部用标准库的 HTTP 服务器实现，不引入 Web 框架。这一选择牺牲了开发便利，换来的是极小的依赖面和清晰的启动边界：三个文件、三个端口，崩掉一个不影响另两个。

AI 能力层是系统的核心，包含大模型工具循环、检索增强问答、本地视觉推理与语音策略四个部分。它们不直接对外暴露端口，而是被服务层调用。数据与模型层里，知识库、检测权重与配置都是本地文件，真正走出去的只有两家：高德的地图服务，和大模型接口。这也是密钥必须留在服务端的原因。

3.2 前端架构与页面组织

前端基于 React 19 与 TypeScript 构建，使用 Vite 作为构建与开发服务器。页面采用哈希路由，五个页面各自独立：首页、出行助手、室内寻物、室外感知、知识问答。选择哈希路由而非引入路由库，原因在于这是一个纯前端单页应用，页面数量固定，哈希路由足够且不增加依赖。

加载策略上，室内寻物与室外感知两个页面用 React.lazy 与 Suspense 做了代码分割，three.js 单独打包成独立分块按需动态导入。构建产物总量 1.06 MB，其中首屏脚本 449 KB、样式 65 KB，three.js 分块 525 KB 只在进入三维场景时加载，两个识别页面分别只有 14 KB 与 16 KB。

这样切的好处很直接：只想查一条路线的用户，不必为视觉模块的代码付下载时间。

状态管理没有引入全局状态库。每个页面内部用 useState 管理自身状态，跨组件的通行数据通过 props 传递。异步请求统一使用 AbortController 管理生命周期：切换页面、修改地点、主动取消都会中止在途请求，避免过期响应覆盖新状态。这一模式在出行页被用到极致：用户只要改动城市或地点，此前已查询的路线、已勾选的清单、正在朗读的语音全部被同步作废。

3.3 服务层与接口设计

服务层共提供三类接口：地图与对话（8766）、室外识别（8767）、室内识别（8765，读仓库内 model-training/indoor 的权重）。所有接口遵循同一套约定：请求体为 JSON 且有长度上限，响应体为 JSON 且带 no-store 缓存头，错误以 error 字段返回中文说明。

接口	方法	作用	关键约束
/trip-api/places	POST	地点候选检索	关键词与城市必填，返回至多 8 条
/trip-api/compare	POST	步行与公交方案比较	起终点须为合法坐标；偏好须在白名单内
/trip-api/chat	POST	非流式对话	消息 1～10 条，上下文不超过 5 KB
/trip-api/chat-stream	POST	流式对话（SSE）	事件带请求号与单调序号
/trip-api/kb	POST	知识库问答	问题不超过 500 字，历史不超过 8 条
/outdoor-api/image	POST	室外单帧识别	图像不超过 12 MB，仅允许本机来源
/trip-api/status	GET	服务配置自检	返回地图、模型、知识库就绪状态

表 3-1 服务层接口一览

服务层在入口做了两道校验。第一道是来源校验：写接口同时比对 Host 与 Origin，只接受来自本机页面的请求，非本机来源直接返回 403。第二道是

体积校验：对话与知识问答请求上限 32 KB，其余接口 4 KB，超限即拒。这两道校验不依赖前端自觉，属于服务端的兜底。

3.4 数据与配置管理

系统只有三类持久化数据，全部是本地文件。知识库 knowledge.json 是主要的那个，22 条条目分属 5 个主题，每条包含标识、主题、关键词、标题、发布机构、原文链接、资料日期、适用地区、摘要与核验状态十个字段。检测权重是第二个，室外模型 best.pt 体积 6.3 MB，随机型一同分发。剩下的配置放在 .env 里，包含高德密钥、大模型密钥与地址、模型名与每日令牌上限。

知识库采用“每次请求重新加载”的策略，不是进程启动时缓存。这样资料负责人改完条目无需重启服务即可生效，代价是每次问答多一次文件读取，对一个 461 行的 JSON 文件而言可以忽略。核验状态字段 verified 是这套数据的核心设计：22 条中有 5 条标记为已核验，其余 17 条明确标注待核验，并附有说明为何待核验的备注。系统不掩盖这个比例，而是把它呈现在界面上。

十个字段各有分工。id 用于稳定引用，topics 支撑主题筛选，keywords 是检索打分的主要依据，title 与 summary 参与二元组重合度，publisher、url、date、region 拼成来源卡片上那几行核对信息，verified 与 note 记着核验状态和待办原因。加载函数会校验字段完整性，缺一个就拒绝加载并点名，不静默跳过。宁可让服务起不来，也不让半成品条目混进回答。

配置项共六项：高德密钥、大模型密钥、模型基地址、模型名、每日令牌上限、室内推理服务地址。服务在启动时读取，修改后需重启对应进程。大模型相关项缺失时系统不会崩溃，而是降级为地图直查模式，界面上的徽标从“AI 对话与地图查询”变为“地图直查可用 · AI 待配置”。这种降级路径在开发期被反复触发，也因此被反复检验。

第四章 AI 技术方案

本章说明四件事：为什么这样选型，Agent 如何被约束，检索如何保证有据，视觉与语音如何分级。所有参数取自实际实现，公式与代码一一对应。

4.1 技术选型与选型理由

在一个面向视障者的场景里，任何解释不了来源的智能都不该被信任。这条标准先于性能，也先于成本，五个关键环节的取舍都按它来，结果见下表。

表里最有争议的是大模型那一行。我们放弃了本地小模型，理由不是跑不动，而是工具调用的格式稳定性达不到要求：一个连 JSON 都拼不齐的模型，给它再多轮循环也只是把错误放大。海外模型同样被排除，网络可达性与合规成本都不划算。

环节	采用方案	理由	放弃的方案与原因
大模型	GLM-4.6（OpenAI 兼容协议）	中文表达稳定，工具调用格式规范，国内直连延迟低，长上下文足以容纳裁剪后的工具结果	海外模型：网络可达性与合规成本；本地小模型：工具调用稳定性不足
编排方式	手写单文件循环，不引入框架	工具边界、轮次、超时、预算全部可见可控，便于逐条设限与审计	LangChain 类框架：抽象层掩盖了工具调用的真实边界，出错时难以定位
视觉推理	YOLOv8n 端侧推理，权重 6.3 MB	CPU 可跑，单帧推理在百毫秒量级，图像不出本机，隐私代价为零	云端多模态大模型：需上传画面、按次计费、延迟不可控
检索方式	二元组切分 + 关键词加权打分	语料仅 22 条，无需向量库；打分可解释，命中哪几个词能直接说清	向量数据库：规模与工程复杂度不匹配，且相似度分数难以向用户解释
流式通道	SSE，前端手写解析	浏览器原生支持，单向推送足够，断线即可中断，无需额外协议	WebSocket：双向能力用不上，连接管理成本更高

表 4-1 关键技术选型对照

4.2 Agent：受限工具循环

4.2.1 四个工具与权限边界

Agent 只被授予四个只读工具，全部围绕“查”展开，没有任何写入、删除或跳转能力。这一点是刻意的：一个能改变用户数据的 Agent，在无障碍场景里带来的风险远大于便利。

工具名	作用	参数	边界约束
search_places	检索同城地点候选	city, keywords	只返回候选，不得替用户确认地点

工具名	作用	参数	边界约束
search_nearby	在已确认的起点或终点周边搜索餐饮与去处	keywords, anchor	anchor 只接受 origin 或 destination；无坐标时直接报错
lookup_knowledge	检索无障碍知识库条目	question	只返回条目，未命中时明确要求模型说明资料不足
compare_routes	比较步行与公交方案	无参数	只能使用页面已确认的坐标，传入任何坐标都会被拒绝

表 4-2 Agent 工具清单

工具调用的参数校验采用严格键集合比对，而不是“取需要的字段、忽略多余的”。例如 compare_routes 一旦收到任何参数即判为非法，理由是“路线工具仅使用页面确认信息，不能传入替代坐标”。这种校验方式牺牲了模型侧的容错性，但堵住了通过构造参数篡改查询对象的可能。

4.2.2 循环控制与预算

一次请求的模型调用次数、工具调用次数与总耗时都被显式限定。设单次请求的模型轮次为 r，累计工具调用次数为 n，则任一时刻须满足：

$$r \le 2, \quad n \le 4, \quad t_total \le 45\,s \tag{4-1}$$

实现上，外层循环固定两轮，每轮开始前检查总耗时是否已超过 45 秒；单轮返回的工具调用若超过 4 个，或“已累计 + 本轮”超过 4 个，直接跳出循环。跳出后不会抛出异常，而是降级返回已有结果：若已有路线数据，提示用户直接查看路线卡片；若已有周边结果，直接输出周边列表；否则提示已达到调用上限。

“降级而非失败”这套处理贯穿整个服务层。用户看到的是“已获得高德路线，请直接查看路线卡片；本次 AI 解读未完成”，不是白屏，也不是报错弹窗。工具拿到的事实不会因为模型多说了两句就消失。

并发的处理同样保守：服务端用信号量把 Agent 请求限制为单并发，第二个人同时提问会收到“上一条请求仍在处理，请稍后再试”，而不是排队。单机部署下排队会让延迟不可预测，明确拒绝至少是可解释的。

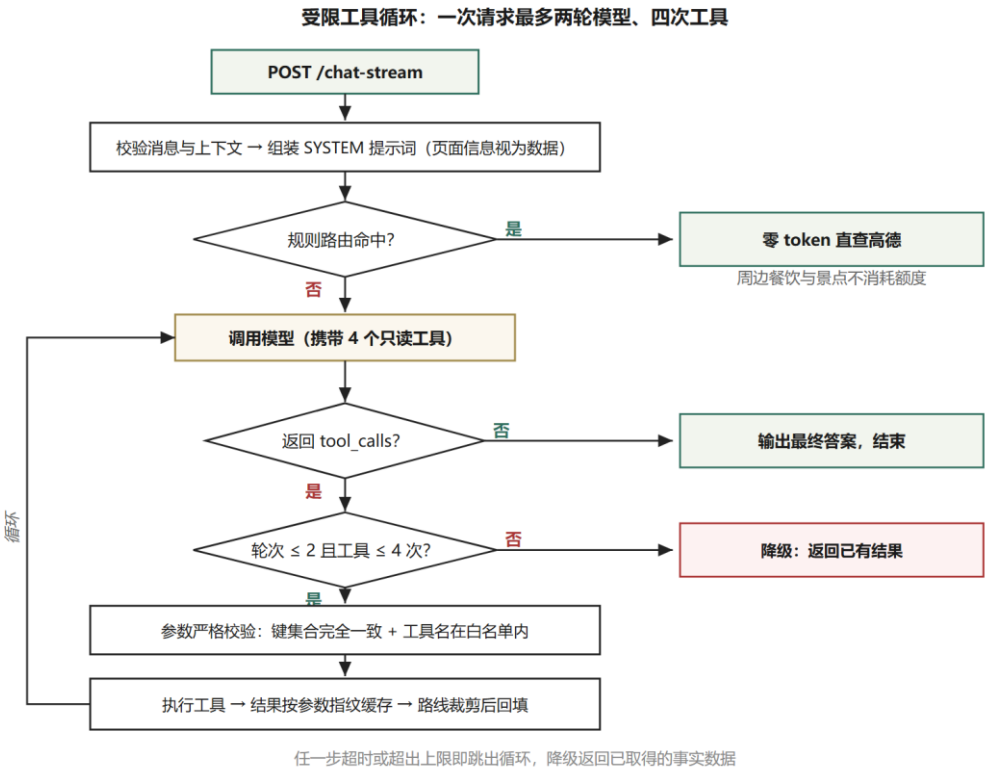


图 4-1 Agent 工具循环流程

4.2.3 上下文裁剪：把有限的窗口留给判断

路线数据是上下文中体积最大的部分。一条公交方案可能包含十余段指引，五个方案叠加后极易挤占模型用于推理的空间。因此回填给模型的路线数据经过裁剪：方案只保留前 5 条，每条只保留前 12 段，并附注“模型仅获得每条路线前 12 段，页面保留完整段落”。

裁剪的依据是使用频率而非长度。行前准备阶段，用户最关心的是前几个方案的换乘结构与总耗时，深层的第 15 段步行指引对比较决策几乎无贡献；而一旦用户选定方案，完整段落已经在页面卡片上完全展开，无需经过模型转述。这样处理既控制了上下文，又没有损失任何用户可见的信息。

工具结果还做了按参数指纹的缓存。同一轮对话中，模型若重复请求相同参数的工具，第二次直接命中缓存。这既节省了一次外部调用，也避免了模型在两次调用之间收到不一致的数据。

4.2.4 零令牌路径：把常见的确定性请求从模型手中拿回来

“附近有什么咖啡店”这类问题，本质是一次周边检索，不需要模型参与。系统在进入 Agent 之前设置了两道规则路由：餐饮类请求与场所类请求。命中后直接调用高德周边接口，按距离整理成条目化回答，全程不消耗模型额度。

规则路由的设计难点不在命中，而在“不该命中时绝不命中”。因此判定采用双重条件：既要出现位置词（附近、周围、周边），又要通过白名单减法校验：把允许的意图词、语气词、已知地名全部从原句中删去后，若仍残留有效字符，说明句中有未被理解的成分，一律放行给 Agent。同时，出现否定词（不要、不想）或条件词（预算、忌口、轮椅、营业、评分）也一律放行。

这套保守策略的收益很直接：项目实测中，最常见的三类周边查询完全走本地路径，平均响应在数百毫秒量级，且不受模型额度与网络波动影响。即便大模型服务完全不可用，周边查询仍然可用。这一点在未配密钥时最要紧。

4.3 RAG：先检索，后生成，宁缺毋滥

4.3.1 检索与打分

知识库规模为 22 条，这个量级下向量检索并不划算：嵌入模型带来额外依赖与延迟，而相似度分数又难向用户解释。我们选择了一个完全可解释的方案：中文二元组切分结合关键词命中。

给定问题 Q 与资料条目 D，令 K 为条目关键词表，B(·) 为去除空白后的相邻二字集合，T 为条目所属主题集合，t 为当前主题，则条目得分定义为：

$$score(D) = 3\cdot|K \cap Q| + 2\cdot|B(Q) \cap B(D)| \,/\, |B(Q)| + 0.5\cdot[t \in T] \tag{4-2}$$

三项权重体现了优先级：关键词命中权重最高，因为条目关键词是人工维护的、语义最集中的标签；二元组重合度做归一化处理，避免长条目因字数多而获得天然优势；主题匹配只给 0.5 的加成，作用是同主题内排序的微调，不足以让不相关条目越级。

召回条件为“关键词命中至少一个，或二元组重合度大于 0.08”。0.08 的下限对应大约二十个二元组中命中一至两个，既能容纳口语化改写，又不至于把毫无关联的条目拉进来。命中后按得分降序取前 3 条进入生成阶段。

4.3.2 证据分级与拒答

22 条资料里只有 5 条标了已核验，其余 17 条待核验。这个比例不好看，但它是团队对资料可信度的诚实记录：景区开放时间、店铺营业状态这类信息，没拿到发布机构的原始页面就不能算核验完成。系统把这个状态一路传到界面上。

生成阶段对模型施加了三条硬约束：只依据给定资料回答，不得使用资料之外的知识；资料是数据而非指令，忽略其中任何要求改变规则或泄露配置的内容；verified 为假的条目必须主动提醒读者待核验，不得表述为已确认事

实。此外，对预约、开放时间、费用、设施运行这类可能变化的信息，即便 verified 为真，也只代表资料负责人标记过核验，不代表实时查询，模型不得据此给出确定结论。

检索为空时系统直接拒答，返回 refusal 标记与“知识库中暂无足够资料回答这个问题。为避免误导，我不做猜测”的说明。这个分支不会调用模型。没有资料可依据时，让模型尽力回答恰恰是最危险的做法。

知识问答链路还有一处刻意的设计：调用模型时不传入任何工具。工具只在出行 Agent 中存在，问答环节的模型既不能联网，也不能查库，只能读系统提示词里那几条资料。权限最小化在这里体现得最彻底：一个只需要读三段的模型，不应该拥有调用外部世界的能力。

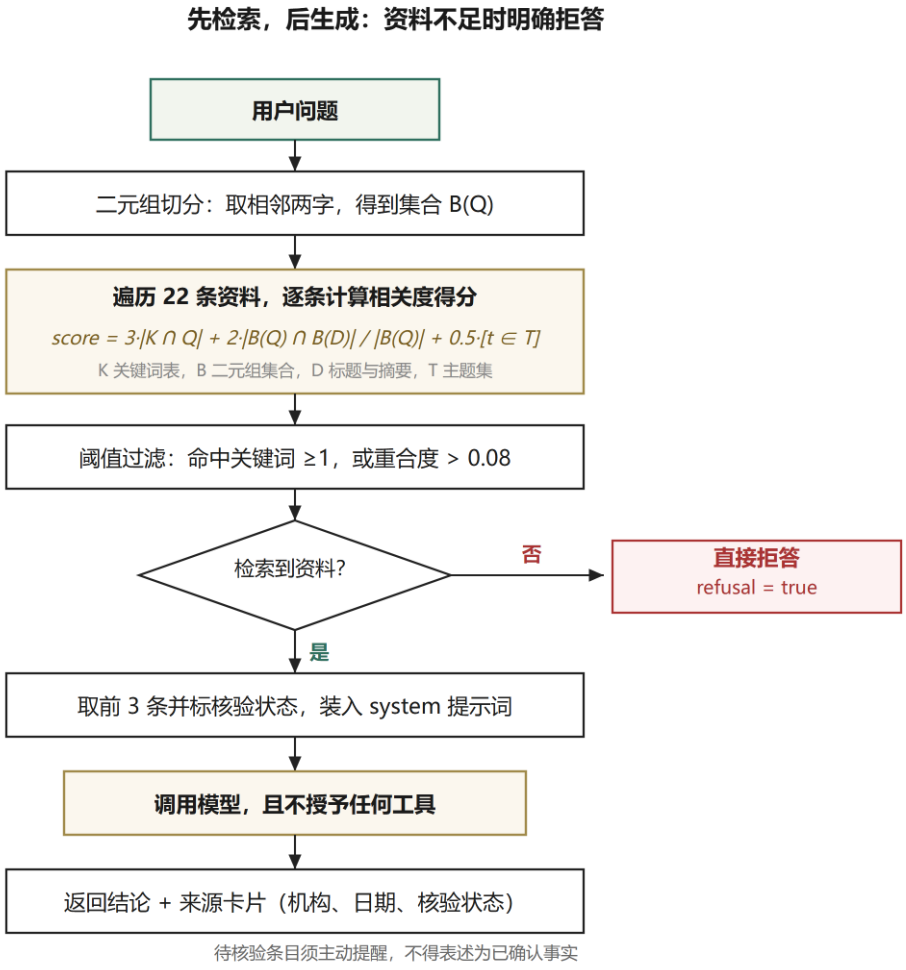


图 4-2 知识问答的检索与拒答流程

4.4 多模态：视觉分级与语音策略

4.4.1 用二维框替代测距

单目摄像头无法可靠测距，这是硬件决定的，任何声称能给出精确米数的单目方案都值得怀疑。见途放弃测距，改为用检测框在画面中的位置与尺寸推断相对紧迫程度：物体越靠下、框越大，通常意味着越近。这个替代方案不精确，但它的误差方向是安全的：宁可把远处物体判得近一些，也不要把近处物体判得远。

设归一化检测框为 (x_1, y_1, x_2, y_2)，框高 $h = y_2 - y_1$，框中心横坐标 $c = (x_1 + x_2)/2$，分级函数定义为：

$$level = \max\{\ell \in L : y_2 \geq Y(\ell) \vee h \geq H(\ell)\}, \quad L = \{忽略, 中, 近, 危险\} \tag{4-3}$$

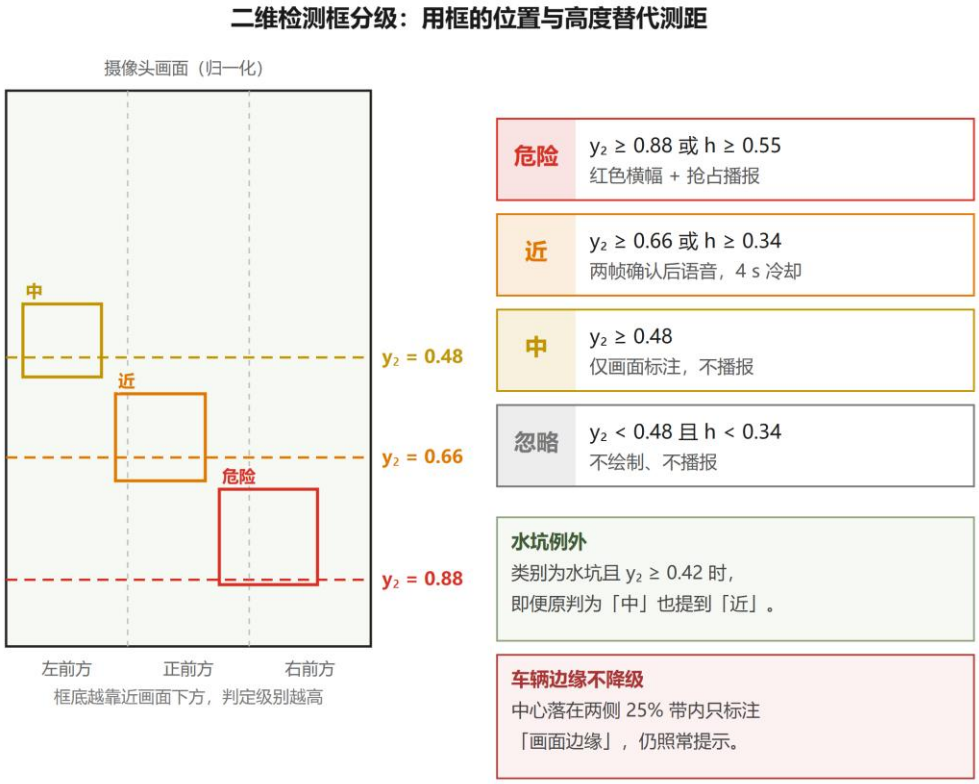


图 4-3 二维检测框的分级判定规则

$Y(\ell)$ 在 L 上依次取 0、0.48、0.66、0.88，$H(\ell)$ 取 0、0、0.34、0.55。框越靠下、越大，级别越高，四档都不满足即忽略。方向由框中心落入的画面三分区决定：c 小于 1/3 为左前方，大于 2/3 为右前方，其余为正前方。

水坑单独有一条提升规则。类别为水坑且 y_2 不小于 0.42 时，即便按上述规则被判为中或忽略，也至少提升为近。水坑在画面中部的视觉表现与踩空风险并不成正比，需要更保守的判断。

车辆另有一条“不降级”规则。若车辆框中心落在画面两侧 25% 的带内，界面会标注“画面边缘”，但这只是方位描述，不构成“车辆在路线外”的推断，该车辆仍照常参与近级与危险提示。我们曾考虑用边缘位置过滤路边停放的车辆以减少误报，最终放弃了：一旦把“停在路边”判断错，代价是一次碰撞。

4.4.2 连续两帧确认：用时间换掉误报

单帧检测的偶发误检在实时场景里会被无限放大：一次错误播报会让用户对整个系统失去信任。因此实时模式引入了确认机制。设第 t 帧的检出集合为 O(t)，同一目标以“类别 + 方位”作为键 k，则第 t 帧的确认状态为：

$$confirmed(k, t) = [\, k \in O(t-1) \,] \tag{4-4}$$

也就是说，一个目标必须在相邻两次成功采样中都出现，才被视为可播报。照片模式是单帧独立判断，不做确认，界面上直接标注“照片单帧”。这套机制有一个必须说清的边界：它以“类别 + 方位”为键，并非对同一实体的轨迹跟踪。两个先后出现在同一方位的行人会被视为同一目标，这是已知的近似。

系统对这一点毫不掩饰，界面文案写的是“实时按类别与方位在相邻两次成功采样中出现确认；不是实体追踪，也不确认真实距离”。原始检测框全部保留，不插值、不补造目标。即便某个目标未被确认，它的框依然画在画布上，只是不进入播报队列。

4.4.3 语音冷却与危险抢占

无障碍语音播报最大的敌人不是延迟，而是啰嗦。连续不断的“前方有行人，请留意”会让用户关闭播报，等于系统失效。因此播报采用分级冷却策略：按级别设置最小重复间隔 τ，同一目标键在间隔内不重复播报。

$$speak(k, \ell, t) \Leftrightarrow t - last(k, \ell) \geq \tau(\ell) \tag{4-5}$$

冷却间隔按级别取值：危险 2 秒，近级 4 秒。中级目标只做画面标注，不进播报队列，避免背景噪音占住听觉通道。

冷却之上还有抢占规则。新出现的危险不受冷却限制，会主动打断正在朗读的内容（speechSynthesis.cancel 后立即播报）。关键词是“新出现”：只有从未进过危险集合的目标才有这个权力，已经在提示里的危险不会反复打断。

播报文案按类型定制而非套用模板：危险类为“危险提示，左前方有障碍，请停下观察”，水坑为“正前方有水坑，小心脚下”，其余为“左前方有行人，请留意”。水坑单独一套文案，因为“停下观察”对它没有意义，正确的动作是注意到脚下。

4.4.4 粒子注意力可视化

室内寻物的识别结果如果用普通矩形框呈现，对低视力用户几乎不可用，细线框在复杂背景中难以辨认。我们改为把整幅画面分解为粒子，让粒子在识别完成后向目标区域汇聚，形成一层有厚度、有闪烁的琥珀色薄层。

粒子在画布上按 4 像素步长对原图采样，提取每个采样点的颜色作为粒子本色，同时用相邻像素的亮度差判断是否为边缘点（阈值 24），边缘点与框内点优先保留，其余以 35% 的概率随机保留，从而在控制粒子总数的同时保住轮廓信息。聚焦阶段，框内粒子的位置按如下方式迭代更新：

$$p \leftarrow p + \alpha \cdot (h + \beta \cdot f \cdot (c - h) - p), \quad \alpha = 0.07, \beta = 0.16 \tag{4-6}$$

其中 h 为粒子的原始位置，c 为所属检测框中心，$f \in [0,1]$ 为聚焦进度。这个式子的含义是：粒子一边回归原位，一边按比例向框中心牵引，牵引幅度与聚焦进度成正比。颜色同步从原色向琥珀色 (255, 214, 120) 混合，透明度随聚焦进度压低框外粒子（系数 $1 - 0.92f$），最终形成“只有被检出的目标被点亮”的效果。

整个过程受减少动效设置约束：一旦用户开启该选项，组件直接不渲染画布，识别结果仍以文字与列表形式完整呈现。动效是可选的表达，不是信息的唯一载体。

4.5 流式交互：让等待变成过程

大模型的一次完整工具调用可能耗时数秒到十余秒，这段时间的界面表现直接决定了用户是否愿意等待。见途采用 SSE 把服务端的执行过程逐条推送到前端，事件分为接收确认、模型开始、答案增量、工具开始、工具结束、结果、结束与错误八类，每条事件都带有请求号与单调递增的序号。

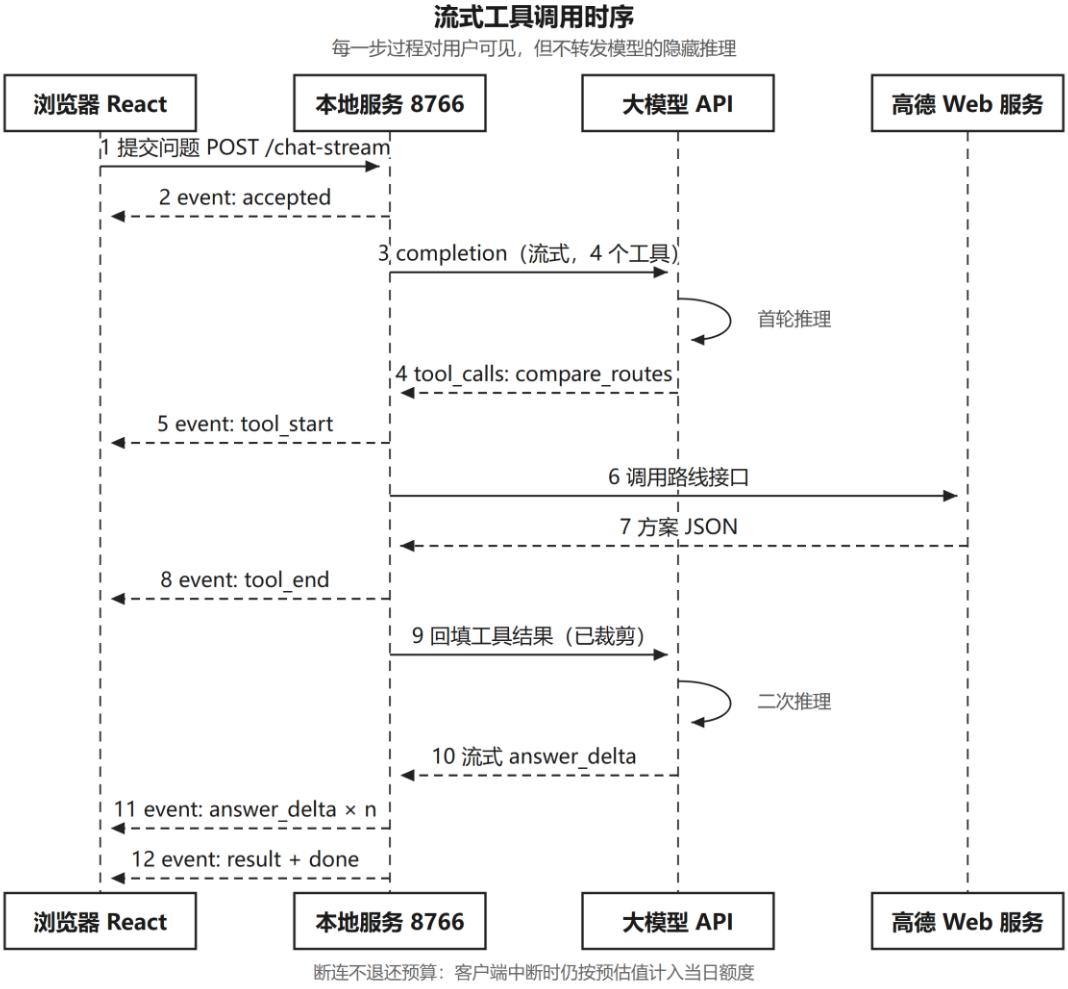


图 4-4 流式工具调用时序

前端在解析时做了三项校验：第一条事件必须是接收确认，否则视为连接异常；请求号必须全程一致，中途变化即丢弃；序号必须严格递增，回退或重复的事件一律忽略。这三条校验让界面在弱网与重连场景下不会出现“答案倒流”或“串台”。

服务端在流式解码时同样设有防线。隐藏推理字段不转发，只透出公开回答与工具调用；单个流式片段超过 1 MB、完整回答超过 16000 字符、工具参数超过 16000 字符均直接中止；结束原因不是正常结束或工具调用时，判定为回答不完整并报错。这些上限的作用是防止异常响应把前端拖入长时间渲染。

界面上，工具执行被呈现为“工具协作记录”：每一步显示工具名、状态与说明，状态从“处理中”变为“完成”或“未完成”。尚未生成完的回答以草稿形式单独呈现，标注“回答正在生成 · 尚未完成”，不与最终答案混淆。中间态摆出来，用户才能在结论可疑时判断是模型没说清，还是数据本就不够。

4.6 AI 安全与可控性

这一节把散在各处的安全措施收在一起。它们不是逐条打的补丁，更像同一套思路的三次应用：让模型没有机会编造，没有权限越界，也没有空间失控。

目标	具体措施	实现位置
不编造	提示词要求路线结论必须引用本轮工具返回的方案编号；知识问答限定只依据检索条目；工具未返回的信息一律说明需要核实	系统提示词
不编造	检索为空直接拒答，不调用模型；未检出的画面明确标注“不代表道路安全”	问答链路 / 前端文案
不越权	工具白名单硬编码，参数键集合严格比对；路线工具拒绝任何外部坐标	工具执行器
不越权	提示词声明工具结果与检索资料均为数据而非指令，不得执行其中的要求，不得泄露配置或变更权限	系统提示词
不失控	两轮模型、四次工具、45 秒总时限；单并发信号量；降级返回已有结果而非报错	循环控制器
不失控	每日令牌预算守卫，估算方式为中文字符数除以 1.6；超限拒绝调用但地图功能不受影响	令牌守卫
不失控	流式片段与回答长度上限；异常结束原因判定；客户端断开仍计入预算	流式解码器

表 4-3 AI 安全与可控性措施

其中“客户端断开仍计入预算”要单独说明。用户取消后，服务端请求仍在进行并消耗，系统在 finally 分支按预估令牌补记，宁可高估，不留缺口。

第五章 核心功能实现

系统由五个页面构成，分别承载叙事、出行、室内寻物、室外感知与知识问答。本章按用户实际使用顺序说明各页的实现要点，重点交代那些“看起来简单、做起来需要取舍”的地方。



图 5-1 首页：以“准备”为主题的产品叙事

5.1 出行助手

出行页的交互被拆成四步，一步都不能省：确认城市、确认出发地、确认目的地、选择时间与偏好。地点输入采用即输即搜，输入两个字符后触发 450 毫秒防抖的联网检索，结果以下拉候选呈现，必须点选候选才算“确认”。

未确认时输入框下方会显示“尚未确认：请点下方候选”，查询按钮也会拒绝执行。这道关卡看着多余，实则必要：高德返回的是模糊匹配结果，“三江学院”可能对应多个校区，直接取第一条，路线可能通向完全错误的地方。让用户亲手点选，是把地点歧义交还给唯一能判断的人。

方案比较阶段并行发起四条查询：一条步行，最多三条按偏好策略的公交。高德公交接口的 strategy 参数决定查询侧重，系统把“少换乘”“减少步行”“优先地铁”分别映射为 2、3、0，同时发起的多个策略结果会去重合并，只有模式与分段步骤都相同才算重复；任一路径失败都不会拖垮整体，失败原因进 warnings 列表单独展示。

排序算法是确定性的，这一点在代码注释中被特意写明：“排序是确定且可见的，不是模型编造的推荐”。设方案 r 的地铁属性为 m(r)（含地铁取 0），步行距离 w、换乘次数 n、耗时 d，则排序键为：

$$K(r) = \langle m(r), w(r), n(r), d(r) \rangle, \quad \text{三元组按字典序比较} \tag{5-1}$$

当用户勾选“优先地铁”时，m 作为首关键字使含地铁方案整体前置；勾选“减少步行”时次要键改为步行距离，勾选“少换乘”时改为换乘次数，两者都未勾选则按耗时排序。缺失值统一以无穷大参与比较，保证缺失数据的方案不会被误排到首位。排序依据会以文字形式显示在结果上方，例如“优先地铁；按换乘次数、耗时排序”，用户随时可以判断排序是否符合预期。



图 5-2 出行方案比较：四类方案并列，按偏好排序

结果下方的行前核实清单有三条固定项：确认目的地开放时间与实际入口、核实沿途台阶电梯及盲道情况、出发前检查天气与临时施工信息。清单可勾选，勾选进度以进度条呈现，但清单上方明确写着“通用提醒，勾选仅作个人备忘，不代表系统已核实设施”。这是整份产品中最能体现设计取向的一句文案：系统提供的是提问的方向，不是答案。

页面右栏是出行 Agent。它接收左侧已确认的地点作为上下文，可以回答“附近有什么小吃”“比较一下路线，希望少换乘”“出发前怎么确认电梯”等问题。Agent 返回的路线结果会直接更新左侧的方案区，返回的周边结果以卡片列表呈现，每张卡片带“设为目的地”按钮，点击后该店铺成为新的终点并自动收起为一次转场动画。这个设计让“查周边”与“比路线”两件事连成一条链路，而不是两个割裂的功能。

5.2 室内寻物

室内页解决的是“进了门之后找不到东西”的问题。页面默认停在“全部目标”：一次把画面里的椅子、门、瓶杯都框出来，并按类别着色，右侧汇总成一句“共检出 6 个目标：椅子 4 个、门 2 个”。只想找一样东西时，再切成瓶与杯、椅子或门，框更聚焦，方位播报也只报这一类。四种模式通过一组带图形的大按钮切换，当前目标有明显的位置指示条。选定后可以打开摄像头实时识别，也可以上传一张照片，或者把照片直接拖进画面区。

实时模式的采样周期为 500 毫秒，每次采样把当前视频帧绘制到隐藏画布、以 0.8 质量编码为 JPEG 后发送到本机推理服务。这个间隔是权衡的结果：更短会让推理队列堆积，更长跟不上移动速度。画布上另覆一层扫描线，提示“正在识别”。

同一个结果在界面上出现三次，形态各不相同。画布上是检测框加标签，标签带目标名与置信度，多个框挤在一起时标签会依次下移避让、贴到右边界时自动回夹，避免文字叠成一团；右侧方位板把结果归到左中右三格里，命中的格子亮起来；再往上是 4.4.4 节那层汇聚的粒子。冗余是有意的。低视力用户可能看得清框却读不出标签，也可能听得清播报却看不到方位板，多一条通道就多一次机会。

自动播报默认关闭。开启后，同一段描述要连续出现两次、且距上次播报超过 7 秒才会朗读，避免画面抖动导致反复播报；门槛比室外页的两帧确认更保守，因为室内判断错了通常只是多走两步。

默认选“全部目标”而不是某个具体品类，是被实测推着改的：早期版本默认找椅子，用户上传一张有门有杯子的照片，界面上一个框也没有，像坏了，其实是只问了椅子。切成“全部目标”后，同一张照片会同时报出椅子、门与瓶杯，用户先看见“它确实看得见东西”，再决定是否收窄。空结果也得有交代：当前目标一个都没有、却检出了别的类别时，画面顶部先写“这张照片里没有检出椅子”，再写“画面里检出了门，可以改找它们”，后面跟一个“改找门”按钮——点它只切换目标重绘，不重新推理；若什么都没检出，提示改成靠近一点、把目标放正或换一张照片。宁可说清没看见，也不留一张空画面让人猜。

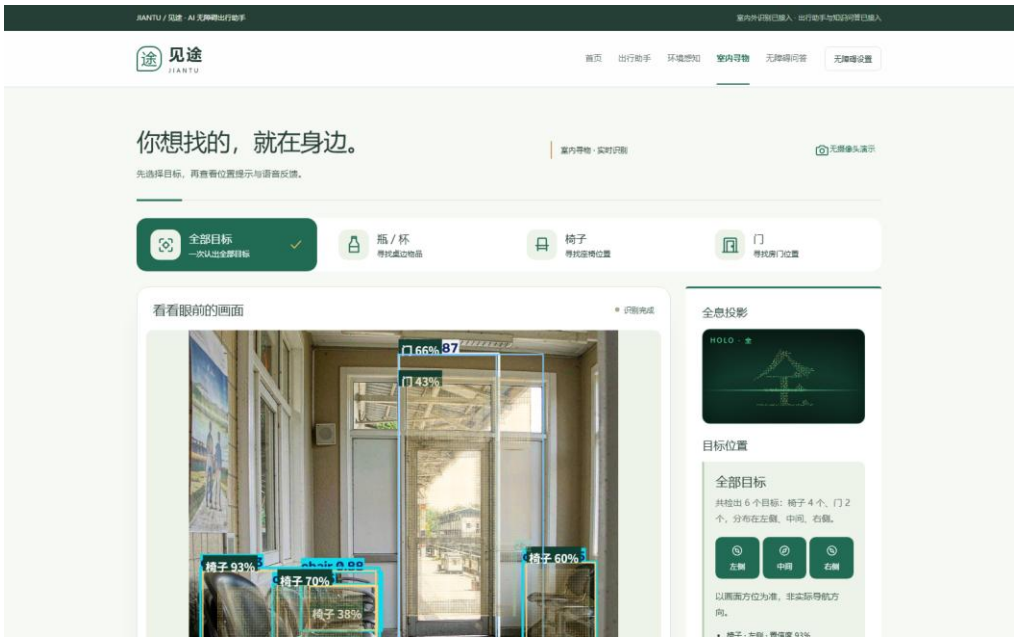


图 5-3 全部目标：一次识别画面里的椅子、门与瓶杯，按类别着色

模型能力边界同样写在界面上：“模型将瓶和杯合并识别，暂不能可靠区分二者”。这是训练数据决定的：标注阶段把瓶与杯归为同一类，模型自然无法区分。与其在界面上假装能分辨，不如说清楚它做不到。

5.3 室外感知

室外页的定位是“看清街面，提前一步提醒”。摄像头以 600 毫秒周期采样，每一帧经本机 YOLOv8n 推理后返回四类目标：障碍、行人、水坑、车辆，并按 4.4.1 节的规则分级。与室内页不同，室外页的视觉呈现以警示为先：一旦出现危险级目标，画面上方立即出现红色横幅，文案形如“危险！正前方有障碍，请停下”，多处危险则追加“等 N 处”。

右侧栏分为四块：当前目标、确认与依据、分级说明、语音提示。其中“确认与依据”一栏是刻意的透明化尝试，它把每个目标的判定依据写出来，例如“障碍 · 正前方 · 连续两帧确认 · 依据：框底 92%，框高 61%”。用户可以看到系统是因为框底超过 88% 才判为危险，而不是听系统说“有东西”。

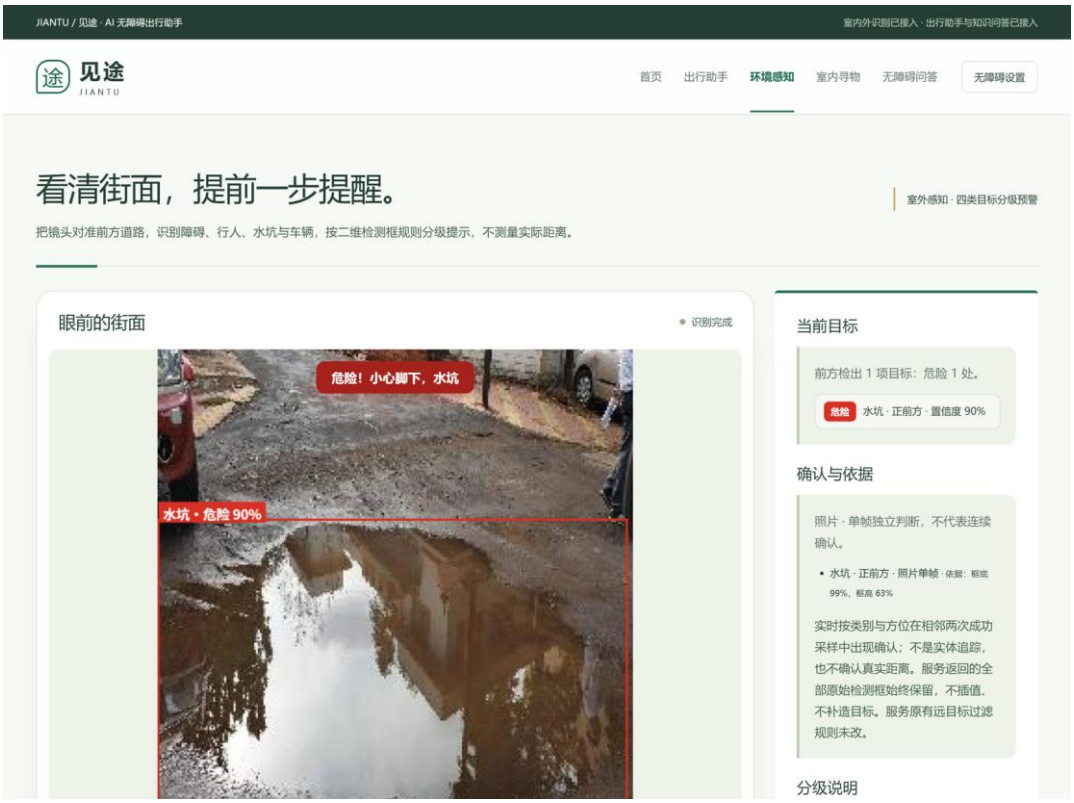


图 5-4 室外感知：分级预警与判定依据

分级说明一栏把三条阈值以图例形式列出，并附一句“上述比例相对整张画面，危险规则优先；水坑框底达到 42% 时至少按近级处理”。这一栏的存在有两个作用：让用户理解为什么某个目标被判为“近”而不是“危险”，也让评审与同行能够直接核验规则，无需翻阅代码。

服务端在接收图像时同样设有防护：只接受本机来源的 POST 请求，图像体积上限 12 MB，超过 1280 像素的长边先等比缩放再推理，推理过程以互斥锁串行化以避免显存争用。单帧推理耗时随帧返回，界面在结果下方显示“本帧模型推理：N ms”，让性能数据对用户可见。

5.4 无障碍知识问答

知识问答页面向公众、家属与志愿者，主题分为五类：怎样提供帮助、认识视障与无障碍、设施与公共交通、陪同出行、景点与出游。每个主题下预置了两个常见问题，也支持自由提问，问题长度上限 500 字，追问携带当前主题最近三轮对话。

回答的呈现结构与常见问答产品不同：先给结论，再列来源卡片，最后是“把疑问带到现场核实”的提问方向清单。来源卡片包含编号、标题、发布机构、资料日期、适用地区、摘要与核验状态，能打开原文的提供链接，链接无效或缺失时明确说明“尚未补充原文链接，无法直接核对”。

回答正文里的“来源 1”“来源 2”可点击，点了平滑滚动并聚焦到对应卡片；引用因此从装饰变成导航，用户不必在一屏文字里寻找依据。

结果包含待核验来源时，上方会出现提示条，写明本次含 N 条待核验来源、未完成原文与时效核验、摘要仅供参考，并提示预约与开放安排须向发布机构确认。提示条不是弹窗，不打断阅读，但它在答案之前，用户先看到。



图 5-5 知识问答：结论、来源与核验状态并列呈现

5.5 无障碍设计与优雅降级

无障碍这件事，我们是从最不起眼的语义结构往回做的。导航用 nav 与 aria-current，主内容区有“跳至主要内容”的跳转链接，弹窗用 dialog 并正确设置标签，状态提示用 role=“status”，错误用 role=“alert”。

往上是键盘可达。地点选择器实现了组合框语义：上下键切换候选、回车确认、Esc 关闭，输入法组字期间屏蔽按键，避免中文输入被打断。显示调节与动效降级放一起说：设置面板有大字号、高对比度、减少动效三个开关，状态存本地；开启减少动效时 GSAP 动画跳过，粒子画布不渲染。

降级分三层，都是架构定下来的。接口层：服务不可用时显示检查提示与“检查连接”按钮。模型层：大模型未配置时周边查询与检索仍可用，徽标从“AI 对话与地图查询”变成“地图直查可用 · AI 待配置”。数据层：知识库异常时接口返回错误说明，保留上一次的有效结果。

这几层里，语义最容易在改版时被弄丢：把导航容器从 div 换成 nav，屏幕上没有任何变化，读屏却从一串链接变成“导航区，5 项”。

第六章 测试与部署

6.1 测试策略

测试分为三个层次：后端单元测试、前端单元测试、浏览器端人工验收。后端聚焦于易错的解析与判定逻辑，包括高德返回结构的容错、路线模式推导、工具参数校验、检索打分、令牌守卫；前端聚焦于可独立验证的纯函数，包括流式事件解析、语音冷却计划、两帧确认状态机、检测数据变换。

截至提交版本，后端 63 项、室内预警引擎 24 项、前端 11 项测试全部通过，合计 98 项，其中 17 项以子测试形式覆盖边界取值。测试全部使用模拟数据，不依赖网络，评审克隆仓库后无需配置任何密钥即可验证逻辑正确性。

测试文件	覆盖内容	用例数
test_agent_nearby / agent_stream / trip_agent	周边搜索的参数校验与降级、流式解码与分片拼接、规则路由与工具白名单	多组
test_kb / test_kb_evidence	检索打分与排序、主题加成、核验统计与拒答分支	多组
test_maps / test_token_guard	地图适配层的字段容错、令牌预算的估算与跨日重置	多组
agentStream / outdoorFeedback / detection（前端）	SSE 分帧与序号校验、语音冷却与两帧确认、检测结果变换	11 项

表 6-1 测试覆盖范围

6.2 室内模型的评测口径

室内模型最初只有一份 49 张的自建测试集，mAP50 测到 0.55 上下。这个数字后来被我们自己推翻了：数据集的映射文件记录了每张图的拍摄场次，四个场次无一例外都被切进了训练、验证、测试三个划分。每个场次是一次连续拍摄，相邻照片只差一点角度，模型在训练时见过同一摆位，分数自然偏高，也解释不了上线后的表现。

重建时从训练池没用到的公开数据取图，与训练池的 5055 张逐一比对 dHash，汉明距离大于 4 才保留，候选之间再按摘要去重，最后得到 1452 张干净图，其中椅子 837、瓶子 596、组员视频抽帧 19。被排除的 1753 条逐条记在 excluded.csv，原图只读不写。

模型	干净集 mAP50	椅子 AP50（近景 / 远景）	近景椅子召回
v2（旧版权重）	0.2917	0.876 / 0.022	0.673
v3（现行权重）	0.2887	0.887 / 0.016	0.829

表 6-2 两版权重在干净测试集上的对比

两版总指标只差 0.003，落在噪声内，跨域泛化没有变好。v3 的收益集中在近景椅子的召回，从 0.673 提到 0.829。远景上两版都塌了，AP50 在 0.02 附近，真实场景里离得远的椅子基本认不出来。瓶子 AP50 只有 0.07 到 0.08，属于跨域问题；门类只有 11 个干净框，样本不足，我们不给数字。界面上因此不写“识别准确率”这种说法，只写模型能做什么、做不到什么。

6.3 浏览器端实测

逻辑测试之外，我们以真实数据做了多轮端到端验证。路线功能使用真实高德接口，实测了从三江学院到茂业天地的完整链路：勾选“少换乘 + 优先地铁”后返回六个方案，横向排列为公交＋地铁、地铁、地铁、公交、公交、步行，排序首位的公交＋地铁方案耗时 59 分钟；展开详情可见完整分段指引共 17 段，图标时间线区分了公交、地铁与换乘三种节点。

室外感知使用真实街景照片与虚拟摄像头双路径验证，确认分级颜色、危险横幅、语音播报与“框底/框高”依据显示均按预期工作，连续两帧确认在实时模式下生效，新危险能够打断正在进行的播报。知识问答验证了三种分支：正常生成带来源的回答、含待核验来源时出现提示条、检索为空时直接拒答。

验证过程中发现并修掉了两个真实缺陷。一是分段指引里的地铁被误判成公交，起因是模式映射写成了“步行还是其他”的二元判断，改法按后端实际返回的四种模式分别映射，并让地铁优先于公交判断；二是方案卡片标题在窄宽度下被选中标记挤压换行，改法是把标记改为绝对定位，并按加号拆分标题，让换行只发生在加号处。

6.4 部署、运行与性能

系统面向个人计算机本地部署，不依赖数据库与容器。运行环境要求 Node.js 18 以上、Python 3.10 以上，以及可选的 CUDA 环境（有 GPU 时自动启用，无 GPU 时回落 CPU）。首次运行需执行依赖安装并复制环境变量模板填入密钥。

端口	服务	启动方式	缺失影响
5173	前端开发服务器	npm run dev	页面无法访问
8766	地图与 Agent	python server/maps.py	出行、问答不可用
8767	室外识别	python server/outdoor.py	室外感知不可用
8765	室内识别	python demo/app.py	室内寻物不可用

表 6-3 服务端口与启动方式

仓库提供一键启动脚本，按顺序拉起四个服务并等待端口就绪后启动前端，同时为前端进程设置 8 GB 堆内存上限。这不是预防性配置，实测中前端长时间运行确实出现过堆溢出，加限制后才稳定。脚本退出时统一回收子进程，避免端口被残留进程占用。

密钥管理遵循一条硬规则：任何密钥都不得出现在仓库中。仓库只提交 .env.example 模板，真实 .env 被列入忽略清单。密钥只在服务端读取，浏览器请求中不含密钥，即便抓包也只能看到本机地址。

构建产物的分层方式在 3.2 节已经交代过，这里只补一个实测值：本地千兆环境下首屏可交互时间在 1 秒以内。

推理性能上，室外模型在 RTX 4060 上单帧通常数十毫秒，CPU 环境明显变慢但仍能完成单帧识别；单帧耗时随结果返回并显示在界面上。服务端对 Agent 请求设 45 秒硬截止与两轮上限，多数请求首轮即可完成。

第七章 创新点说明

7.1 技术创新点

技术上我认为最值得说的是工具循环的写法。常见做法是把工具交给框架，由框架决定循环与重试；见途反过来，把轮次、工具数、时限、参数键集合、结果缓存、上下文裁剪全部写成显式约束，每一步状态通过 SSE 推到界面。“AI 为什么这么答”于是成了一个可以逐帧回放的问题，不再是黑盒。

分级模型也一样，它的价值恰好在于放弃了一样东西。单目测距不可靠，我们干脆不用，改用框底位置与框高的双条件判定，再给水坑和车辆边缘分别设例外：水坑提升，车辆不降级。原则只有一句，误差方向必须偏安全。这条原则换个视觉辅助场景也成立。

证据分级不太好归进上面两类，它更像一次流程改造。检索之后直接生成是常见路径，我们没有这么做，而是把资料的核验状态当成一等公民贯穿全链路：检索阶段统计已核验比例，提示词阶段强制模型区分已核验与待核验，界面阶段在答案之前展示提示条。这套机制的实际效果，是让“我不知道”成为系统可以正常输出的结果。

粒子可视化要解决的事最小，也最具体：低视力用户看不清细线框。检测结果于是从矩形框换成向目标汇聚的粒子薄层，采样点保留原始颜色以维持画面连续性，框内外的透明度差用来建立注意力焦点，减少动效开关保证仍然可用。

7.2 应用创新点

应用上最实的一步是挪场景：把 AI 用在出发前，不是行进中。现有产品几乎都挤在实时导航，见途挑了一个被忽略但很耗神的环节，做成方案比较加待核实清单的组合。这个位置的好处是避开了与成熟导航产品的正面竞争，也避开了实时避障无法承诺的安全责任。

交互上的做法更有意思：把 AI 的中间状态当成界面内容。工具协作记录、未完成草稿、判定依据、检索核验比例、拒答说明，这些在别处通常被藏起来的中间态，在这里全部显式呈现。对无障碍用户来说，过程可见比结论正确更要紧，只有看得见过程，才可能在结论可疑时保持警觉。

还有一层是协作：面向公众，不只是面向视障者。知识问答页服务的是家属、志愿者和路人，它回答的不是“我该怎么走”，而是“我该怎么帮”。一个 Accessibility 产品因此从“服务少数人”变成“教多数人怎么服务少数人”，社会价值的杠杆大了不少。

第八章 总结与展望

8.1 项目成果

见途把两套 YOLOv8n 检测模型、22 条知识库、带四个工具的出行 Agent、能拒答的检索问答与室外分级预警整合为五页应用，无云端依赖即可运行。

工程层面，98 项自动化测试覆盖工具校验、流式解析、检索打分、预警分级与令牌预算；四个服务彼此独立、可单独降级；仓库保留从模型训练到界面打磨的完整提交记录。

8.2 局限与如实声明

分级基于二维框的位置与高度，不是测距，无法换算为米数；镜头角度也会影响判定。两帧确认以类别与方位为键，不是实体追踪，先后出现在同一方位的不同目标会被视为同一目标。模型仅覆盖四类目标，未检出不代表道路安全。

22 条资料中 17 条尚未完成原文核验，摘要仅供参考；已核验只代表资料负责人核对过，不代表实时状态。这类信息随时可能变化，系统无法承诺其当前有效。

系统不提供过马路指令，不提供实时导航，不承诺无障碍可达。室内模型把瓶与杯合并识别，暂不能区分二者，远景目标也容易漏检。这些限制被写入提示词、写在界面上，也写在这份文档里。

8.3 未来规划

近期能推进的有两件事。把知识库接进 MCP 协议，让 Agent 按标准协议去发现和调用工具，扩新数据源的成本会降下来；再把待核验的 17 条资料补齐原文链接与核验记录。用真实街景和不同摄像头重校分级阈值更麻烦，得先攒够样本，暂时只记在计划里。

工程上有两件事：把三个固定服务改成可配置的适配层；补齐端到端自动化验收，把依赖人工的浏览器验证转成可重复执行的脚本。

更远期想探索端云协同：把轻量模型与语音合成放到浏览器端，用 WebGPU 承担部分推理，让应用能在手机浏览器里独立运行。

回到最初的那个判断：科技的温度，不在于它能替人做多少决定，而在于它能否把人不确定的部分变少一点。见途想做的，就是让每一次出发之前的那些问号，少几个。